Android/iOS Screenshot Testing Strategy

1. Recommendation and quality objectives

News feeds combine long German headlines, imagery, urgency banners, themes and empty or error states, and frequent releases make exhaustive manual visual checking impractical. The objective is trustworthy pull-request feedback on unintended visual change without a slow or flaky automation bottleneck.

Recommendation: a hybrid approach. Share scenario contracts, fixtures, comparison and review policy across platforms. Use fast native component snapshots where the production stack supports them, plus a small real-app device suite for integrated rendering, with human review of every reference change. Production tool choices follow inspection of the actual codebases; this strategy assumes neither React Native, Compose nor SwiftUI, nor any knowledge of the employer's internal architecture.

2. Demonstrated Android architecture

The working demo runs on Android only (Pixel_10/API 37 emulator). It uses the adapted open-source Volksverpetzer React Native/Expo app, not employer code.

- **Deterministic data:** development-only (`__DEV__`) in-app fixtures supply synthetic articles through the app's existing WordPress mapping into the real `Feed`, `ArticlePost` and `EmptyComponent` UI, with a bundled image and fixed text. **Appium drives and captures the UI; it does not provide the mocked data — the app's fixture seam does.**
- **Scenario selection and isolation:** each test terminates the app and cold-launches it through the existing Expo development-client deep link with `qa_scenario=<id>`. The selection is resolved once, before the UI subtree mounts, and is immutable for that launch. The app exposes an identity marker that the test verifies before capturing.
- **Runner:** Appium 3 (3.3.0) with UiAutomator2 and Python/pytest. A page-object model separates `HomeScreen` from the `ArticleCard`, `BreakingBanner` and `EmptyFeedState` component objects, which own scenario-specific readiness (image loaded, layout measured, successful empty fetch). There are no fixed sleeps.
- **Capture and comparison:** native element screenshots of the real components. The Pillow comparator counts a pixel as changed when any RGB channel differs by more than 12. It fails above a 0.2% changed-pixel ratio or on any size mismatch, and writes actual, diff-overlay and JSON metrics. Full-screen context images are diagnostic only.
- **References:** human-approved PNGs stored per platform and device profile (`baselines/android/pixel10-api37/`). Tests never create or update a golden.

Scenario	Visual risk and captured element	UI provenance
`long_headline`	Long German headline wrapping in an image-bearing teaser; outer card, 1028 × 1237	Real `ArticlePost` with synthetic data
`breaking_news`	Urgency banner layout and long title; banner, 1028 × 493	Development-only banner added to the sample app for this challenge; not an upstream VVP feature
`dark_article`	Dark-theme contrast of the same card; 1028 × 1237	Real VVP dark palette through a nonpersistent development-only theme override
`no_image`	Text-only teaser with no image panel; card, 1028 × 711	Real `ArticlePost`; challenge-scoped QA adaptation for absent media, not verified upstream support
`empty_feed`	Successful zero-item feed response; empty-state message region, 1028 × 309	Real `EmptyComponent`; **not** offline, error or loading coverage; the donation card below is excluded

3. Verified evidence

Local Android (Windows-hosted emulator): all five scenarios passed with zero changed pixels against their approved references; repeated `empty_feed` captures were byte-identical.

Intentional mismatch demonstration: a test marked `intentional_failure` (excluded from default selection) compares the real `breaking_news` banner with a deliberately wrong, separately stored reference (red and blue channels swapped, in `qa_visual/demo_assets/`, never a golden). It fails at the comparison assertion (468,349 of 506,804 pixels, about 92.41%, above the unchanged thresholds) and writes actual, diff and JSON evidence while the normal test still passes. This proves mismatch detection and reviewable output; it is **not** an application regression.

GitHub Actions (manual, advisory, Ubuntu runner, headless API 37 emulator):

Run	What it showed
Build `42616325655`	Debug APK built from commit `d48e8d7` and published as a reusable artifact.
Visual run `42620071563` (commit `d48e8d7`, `visual_scope=breaking_news`)	`breaking_news` passed with 0 changed pixels against the committed golden.
Full suite on the same APK (run ID not recorded here)	4 scenarios (`breaking_news`, `dark_article`, `empty_feed`, `long_headline`) passed golden comparison with 0 changed pixels. `no_image` failed before capture: the app process died with a native `SIGSEGV` in React Native's Fabric renderer on the 16 KB-page image (`mqt_v_js`, `libreactnative.so`). **This is not a passing five-scenario run.** The same signature was seen in an earlier run.

The cause of the `no_image` crash is **unresolved**: an isolated `no_image`-only workflow option exists (commit `79126a9`) but could not be executed because of GitHub platform errors, so fixture-specific, launch-order, image or resource causes are all unproven. The visual workflow is advisory: pytest failures are reported but do not fail the job, so the job status alone never proves a pass.

4. Cross-platform reuse boundaries

Layer	Approach
Shared	Scenario identifiers and versioned fixture contracts, high-level assertions (identity, readiness, content), comparison and reporting format, baseline-review policy and CI templates.
Android	UiAutomator2 driver, locators, launch/reset, fixture adapter and Android goldens per device profile (implemented).
iOS (proposed)	macOS simulator with the Appium XCUITest driver; the same scenario names delivered through an iOS launch-argument or deep-link adapter; separate locators and iOS goldens per simulator/OS. Not implemented or executed.
Fast component snapshots (proposed)	Paparazzi or Roborazzi for Android Views/Compose; swift-snapshot-testing for UIKit/SwiftUI. Only where compatible with the production stack; they do not render arbitrary React Native components.

Use the lowest-cost layer that gives the required confidence (many states as component snapshots, few device journeys) without duplicating assertions.

5. Deterministic data in an existing production codebase

Version a shared fixture schema and scenario identifiers alongside the tests, maintained jointly by development and QA, with thin platform adapters. Inject fixtures at the repository/data-source boundary through dependency injection in a test build variant. Where no such seam exists, redirect the API client to a controlled local mock server with fixed routes and assets. Keep production networking unchanged and verify that release builds cannot enable fixtures.

Each scenario pins bundled images, time and timezone, locale, theme, fonts and font scale, animation policy and session/entitlement state. Reset caches and sessions between cases, wait for explicit readiness signals, and block and report unexpected external requests in test mode.

6. Golden governance and review

Partition references by platform, device size, OS/renderer and display configuration, and commit only approved PNGs. Publish actual, expected, diff and metrics together, including on failure. A missing reference or size mismatch fails; CI never regenerates or accepts goldens automatically. An intentional visual change arrives as an explicit pull request with rationale and comparison evidence. The feature reviewer and designer approve the result, QA checks fixture and profile consistency, and verification is rerun against the new reference before merge. Protect reference directories with code ownership; Git history keeps previous references. Demo thresholds suit one profile, not all.

7. CI/CD, Scrum and risk-based coverage

Implemented: two manually triggered workflows. One builds the debug APK. The other reuses it (after a provenance and compatibility check), boots a pinned emulator, runs Metro and Appium, runs `pytest -m visual` (all five, or one selected scenario), and uploads actual/diff/metrics, Allure, launch-failure diagnostics and logs even on failure. **Proposed, not implemented:** a pull-request trigger. On each pull request: build the test variant with pinned dependencies and versioned fixtures; run affected component snapshots, then a focused device smoke suite; compare against platform/profile references; fail unexpected changes and publish actual, diff, metrics and logs; require human review for reference updates, then rerun verification. Android runners need emulator acceleration and sufficient memory; iOS needs macOS with pinned Xcode and simulator versions. Keep pull-request gates small, using component ownership and change maps. Run a risk-based nightly or pre-release matrix across themes, font scales, device sizes, OS versions, offline/error and premium states; a device cloud is optional for broader hardware. In refinement, developers, QA, designers and product owners agree on critical states. Developers maintain component snapshots, QA owns integration diagnostics and selection rules, designers approve intended visuals, and product owners resolve acceptance questions.

8. Flakiness, scaling and rollout

Pin emulator/simulator images, fonts, assets and rendering settings. Use explicit readiness, isolate each scenario with a cold launch or state reset, and capture element rather than full-screen images so clocks and status bars do not cause diffs. Separate infrastructure and readiness failures from genuine visual mismatches. Allow one logged retry only for diagnosed infrastructure errors, never retry a pixel mismatch until it passes, and quarantine persistent flakes with an owner and deadline.

Roll out with a pilot of a few high-risk components on one device profile, and prove repeatability and baseline ownership before making checks blocking. Then add focused pull-request gates, an iOS pilot and further risk-based scenarios. Pair developers and QA engineers on fixture seams, diagnosis and review, and document examples for other teams. Measure PR feedback time, false positives, escaped visual defects, flakiness, CI cost and manual effort before claiming improvement.

9. Limitations

- Only Android native execution is verified; there is no iOS execution. Hosted CI is manual and advisory, with no required pull-request check, and the full five-scenario suite has not passed on GitHub because of the open `no_image` native crash.

- Goldens were reviewed on a Windows-hosted emulator; the Linux runner matched four of them pixel for pixel, but different emulator or image builds may still drift, so profiles should be pinned and re-approved per environment.
- The harness uses an Expo development client with Metro, so a first-launch Expo developer-menu overlay is dismissed by the test runner; standalone release-build behaviour is not validated.
- The existing combined `pnpm check` fails on a known spelling issue (`FIXTUR`) in a fixture file. A historic one-pixel native layout difference has an unresolved root cause; a reviewed runtime-specific golden was approved instead.
- The `empty_feed` reference excludes the donation card, and the failing demonstration proves mismatch detection, not detection of a code regression. The adapted app carries third-party brand assets whose redistribution rights are unconfirmed (see `NOTICE.md`).